

Arborescence

On a reconstruit l'arborescence pour correspondre à nos besoins.

✓ PROJECT-ROOT	xpl : contient les exemples au format cpp
> build	build : contient les exe des exemples cpp. Il contient aussi le CMakeLists utile à la compilation des exemples.
✓ doc\html	
> build\html	doc/html/build : contient les fichiers issus du build Sphinx
> source	doc/html/source : la source avec les fichiers utiles à Sphinx (les notebooks pythons etc...)
> scripts	
> xpl	scripts : c'est ici que se trouve doc_copyExample.py et doc_generateArchives.py

Lors de la construction de cette arborescence, on a rencontré des issues :

Issue 1 : chemins manquants	Quand on testait les fichiers doc_copyExample, les dossiers xpl et build/bin étaient absents. On avait seulement archive et source.
Issue 2 : noms des exe / pas les bons	Les tableaux corresponding_exe et corresponding_directory ne correspondaient pas avec les exemples obtenus finalement, que l'on voyait dans le dossier media
Issue 3 : marmoteLog qui était absent	marmoteLog n'étant pas considérée comme une cible séparée, cela a engendré des erreurs à plusieurs endroits du code

Générer la documentation avec Sphinx

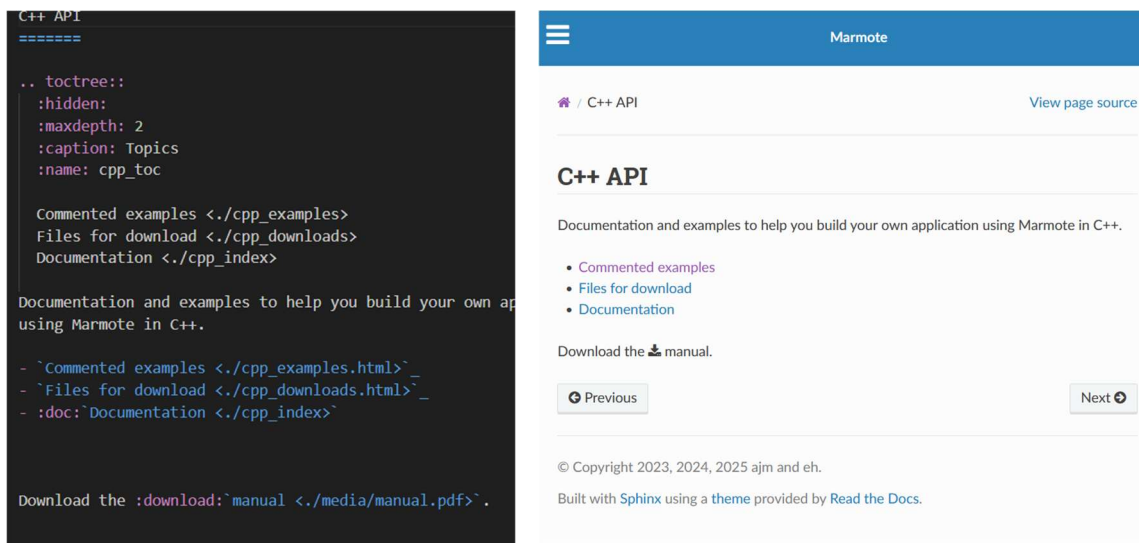
<https://www.sphinx-doc.org/en/master/usage/quickstart.html#running-the-build>

Sphinx est un module Python permettant de générer de la documentation.

Il réutilise les fichiers .rst*, md et notebooks pour les transformer en HTML. Ici, il lit le dossier doc/html/source pour produire les pages.

Il est livré avec un fichier conf.py qui permet de paramétrer l'usage de Sphinx pour notre projet.

*fichier rst : se présente sous cette forme, contient le contenu textuel de la page HTML



On a d'abord installé sphinx sur notre environnement avec les extensions suivantes :

```
python -m pip install -U sphinx sphinx-rtd-theme myst-parser nbsphinx
```

Issue 1 : extension recommon mark manquante	Build interrompu ; installation de l'extension
Issue 2 : pandoc manquant	Build interrompu : installation via conda forge

Modifications de sécurisation

La sécurisation des scripts s'est faite en deux temps complémentaires.

Dans `doc_generateArchives.py`, on a d'abord renforcé la maîtrise du contexte d'exécution en conservant un contrôle strict du répertoire de lancement (logique `scripts/marmote/source`, sinon arrêt), puis on a ajouté un mode `--dry_run` pour simuler la chaîne sans effets de bord, ce qui permet de vérifier les chemins et la séquence avant toute copie, suppression ou déplacement.

```
# ajout assia : vérification de répertoire
# récupérer le répertoire actuel
current_directory = os.getcwd()
print(f"Working directory is : {current_directory}")
# arrêter si pas dans source, scripts, ou marmote
current_name = os.path.basename(current_directory)
if (current_name=="scripts"):
    position=os.path.join("../", "doc", "html", "source")
    os.chdir(position)
    print("Change directory in : ",os.getcwd())
elif (current_name=="marmote"):
    position=os.path.join("doc", "html", "source")
    os.chdir(position)
    print("Change directory in : ",os.getcwd())
elif (current_name=="source"):
    print("No directory change",os.getcwd())
else:
    print("Wrong working directory. Program stops")
    sys.exit()

# Directories paths
print("Beginning of script")

# ajout assia : dry-run
args = sys.argv
generate_only = False
dry_run = False
if ( len(args) > 1 ):
    if ( args[1] == "-h" ) or ( args[1] == "--help" ):
        print(f"Usage: {args[0]} [-h|--help] [-n|--dry_run]")
        print("\tThis script is to be stored in .../scripts\n\tand is to be run from directory .../doc/html/source")
        sys.exit(0)
    elif ( args[1] == "-n" ) or ( args[1] == "--dry_run" ):
        dry_run = True

# ajout assia : dry-run
if dry_run:
    print("Dry run : not executing archive/copy instructions")
```

Dans `doc_copyExamples.py`, la sécurisation a surtout porté sur la partie la plus sensible, c'est-à-dire l'exécution de commandes externes: on a réaligné les exécutables réellement appelés (notamment en Windows avec suffixe `.exe`), corrigé le mapping des binaires pour éviter les appels erronés, traité le cas particulier d'exemple4 (conflit dossier de données `.mcl` / exécutable), et corrigé des incohérences de destination lors des déplacements de résultats. Le processus a été itératif: diagnostic via `--dry_run`, reproduction des erreurs réelles, correction ciblée, puis validation par exécution complète jusqu'à obtenir un flux stable de génération (`.cpp`, `.res`, `.cmd`, CMake) sans interruption. On a également, comme pour `doc_generateArchive`, arrêté le programme si on se retrouve dans le mauvais dossier.

(chercher : # ajout assia)

```
PS C:\Users\assia\Documents\projet_univ\dossier 2\source\source> python ../../Archive/doc_copyExamples.py --dry_run
>>
Working directory is : C:\Users\assia\Documents\projet_univ\dossier 2\source\source
No directory change C:\Users\assia\Documents\projet_univ\dossier 2\source\source
Beginning of script
Source files destination = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\media [Exists]
CMakeFile destination = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\instructions\all_ex [Exists]
Binary = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\..\..\..\build\bin [Does not exist]
Code = C:\Users\assia\Documents\projet_univ\dossier 2\source\source\..\..\..\xpl [Does not exist]
Dry run: not executing copy instructions
PS C:\Users\assia\Documents\projet_univ\dossier 2\source\source>
```

Présentation de Xeus

xeus est une bibliothèque C++ qui implémente le protocole Jupyter. Elle sert de base pour créer des kernels interactifs, notamment xeus-cling pour exécuter du C++ dans des notebooks cellule par cellule. Concrètement, xeus gère la communication Jupyter (exécution, sortie, complétion), et cling fournit l'interpréteur C++ (LLVM/Clang).

xeus GitHub (core) : <https://github.com/jupyter-xeus/xeus-cling>

Spécifications OS

D'après la documentation officielle de xeus-cling sur GitHub, le projet fournit des paquets prêts à l'emploi pour Linux et macOS, mais pas pour Windows à ce stade. La formulation du README est explicite: les distributions sont disponibles sur Linux/OS X, et il est indiqué qu'aucun package Windows n'est fourni pour le moment. Cela explique les difficultés rencontrées en environnement Windows (conflits de toolchain, intégration plus fragile), alors que le fonctionnement est généralement plus direct sous Linux, notamment via conda-forge et WSL.

Sur Linux et macOS, le fonctionnement de xeus-cling est beaucoup plus fluide parce que les paquets officiels sont fournis pour ces plateformes. En pratique, on installe le kernel via conda-forge dans un environnement dédié, puis Jupyter détecte automatiquement les kernels C++ (xcpp17, xcpp20, etc.) et permet l'exécution cellule par cellule dans le notebook. Le point clé est de garder une chaîne cohérente dans le même environnement (Jupyter, kernel et dépendances C++), ce qui limite les problèmes de linkage ou de kernel crash. C'est précisément ce mode d'usage "packagé" Linux/macOS qui est documenté officiellement dans xeus-cling, contrairement à Windows où aucun package prêt à l'emploi n'est annoncé.

Différents types de bibliothèques Xeus

Xeus se décline sous différentes bibliothèques, l'essentiel est de ne pas les confondre. Seul xeus-cling offre un kernel c++. Sinon, on retrouve :

- Xeus : le cœur du framework, qui implémente jupyter
- Xeus-cling : dont le kernel c++ est basé sur cling
- Xeus-python : qui offre un kernel python
- Xeus-sql : kernel sql
- Xeus-zmq : qui est une couche de transport ZeroMQ qu'utilise les kernels
- Xeus robot, xeus lua ...

La base : Xeus et xeus-zmq. Puis les autres kernels sont construits par-dessus.

On garde en tête que pour nos besoins actuels, on utilise xeus-cling.

Comment créer un notebook c++ avec Xeus

Comme rappelé précédemment, xeus-cling ne propose pas de packages prêt à l'emploi sur Windows. On utilise donc le Windows Subsystem For Linux.

On y installe conda.

On crée un environnement appelé xeus-cpp où on installe marmote xeus-cling et jupyter. On clique sur le lien renvoyé en lançant le serveur jupyter.

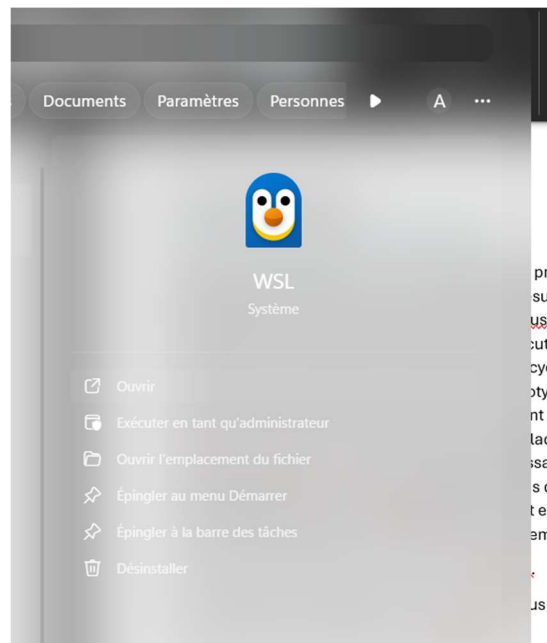
```
conda create -n xeus-cpp-env -c conda-forge python=3.12 jupyterlab xeus-cling -y
```

```
conda activate xeus-cpp-env
```

```
conda install -c marmote -c conda-forge marmote -y
```

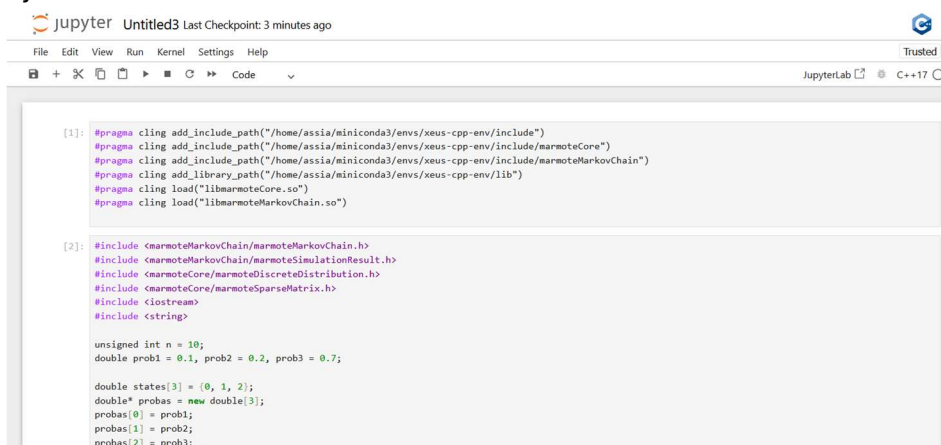
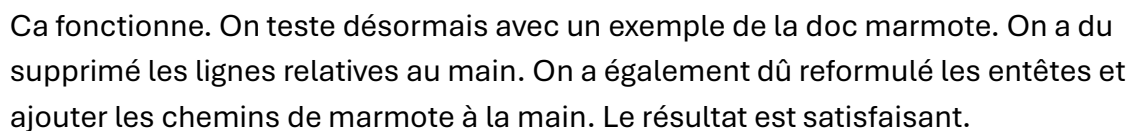
```
jupyter kernelspec list
```

```
jupyter lab --no-browser --ip=0.0.0.0 --port=8888
```



```
(xeus-cpp-env) assia@N15I711-16GR512:~$ jupyter kernelspec list
Available kernels:
python3      /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/python3
xc11         /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc11
xc17         /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc17
xc23         /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xc23
xcpp17       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp17
xcpp20       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp20
xcpp23       /home/assia/miniconda3/envs/xeus-cpp-env/share/jupyter/kernels/xcpp23
```

Le kernel c++ apparait bien. On teste un exemple simple :



```

discrete sparse
3
  0      0 2.500000e-01
  0      1 5.000000e-01
  0      2 2.500000e-01
  1      0 4.000000e-01
  1      1 2.000000e-01
  1      2 4.000000e-01
  2      0 4.000000e-01
  2      1 3.000000e-01
  2      2 3.000000e-01

stop
discrete values { 0 1 2 } probas {    0.1    0.2    0.7 }
Trajectory
  0      1 1
  1      2 2
  2      2 2
  3      1 1
  4      0 0
  5      0 0
  6      1 1
  7      0 0
  8      1 1
  9      2 2
 10      2 2

```

Issue 1 : kernel died	Le kernel mourait lors de l'exécution de code Marmote non résolu (headers/libs non configurés).
Issue 2 : version de libboost incohérente	Marmote (ou une dépendance) attendait une version précise (libboost_thread.so.1.82.0). Si elle manque, les programmes ne démarrent pas
Issue 3 : headers marmote introuvables	Headers Marmote introuvables marmoteMarkovChain.h et dépendances non trouvés au départ dans le notebook.

Alternatives à Xeus-cling

Il existe des alternatives à xeus-cling sous windows. On dénote jupyter-cpp-kernel, qui se base sur g++ :

Doc/PyPI : <https://pypi.org/project/jupyter-cpp-kernel/>

GitHub : <https://github.com/shiroinekotfs/jupyter-cpp-kernel>

Guide install Windows : https://raw.githubusercontent.com/shiroinekotfs/jupyter-cpp-kernel/master/INSTALL_ON_WINDOWS.m

On l'a testé sur l'exemple précédent. Cependant, après installation, les bugs s'enchainent au moment de l'exécution.

```
std::cout << std::endl;

delete simRes1;
delete c1;
delete initial;
delete[] probas;
return 0;
}

In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\random:35:0,
      from C:\Users\assia\miniconda3\envs\jcpp-win\Library\include\marmoteCore\marmotePolicy.h:5,
      from C:\Users\assia\miniconda3\envs\jcpp-win\Library\include\marmoteMarkovChain\marmoteMarkovChain.h:13,
      from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:4:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\c++0x_warning.h:32:2: error: #error This file requires compiler and library support for the ISO C++ 2011
standard. This support must be enabled with the -std=c++11 or -std=gnu++11 compiler options.
#error This file requires compiler and library support \
^~~~~~

In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\postypes.h:40:0,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iosfwd:40,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\ios:38,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\ostream:38,
      from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iostream:39,
      from C:\Users\assia\miniconda3\envs\jcpp-win\share\cpp_header\check_cpp.hpp:23,
      from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:1:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:164:11: error: '::vfwscanf' has not been declared
using ::vfwscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:170:11: error: '::vswscanf' has not been declared
using ::vswscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:174:11: error: '::vwscanf' has not been declared
using ::vwscanf;
      ^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cwchar:191:11: error: '::wcstof' has not been declared
using ::wcstof;
```

Des versions de g++ incompatibles étaient mises en cause, mais après mise à jour de mingw, récupération des dernières versions de g++ utiles pour l'appel à marmote, le kernel ne fonctionnait plus (il se basait sur un mingwe trop ancien pour marmote).

Cette solution pourrait fonctionner pour créer des notebooks, mais pas pour marmote.

Il existe aussi cppyy , mais nous ne l'avons pas retenu ici, car il s'agit davantage de c++ piloté depuis python, sans kernel, et n'est pas adapté au chargement des headers pour marmote.